

Master – BubbleSort & verwandte Themen

(فارسی + Deutsch)

Diese Datei fasst alle prüfungsrelevanten Muster rund um BubbleSort zusammen: Standard-Variante, Objekte, Generics, absteigend, optimiert mit Flag, Fehlerkorrektur, CocktailSort und prüfungsnahe Übungen (GA2-Stil).

این فایل تمام الگوهای مهم مرتبط با BubbleSort را برای امتحان IHK جمع‌بندی می‌کند: نسخه استاندارد، کار با آبجکت‌ها، نسخه جنریک، مرتب‌سازی نزولی، نسخه بهینه با فلگ، اصلاح خطا، CocktailSort و تمرین‌های شبیه GA2.

نسخه استاندارد – 1. Standard-BubbleSort (Integer)

1.1 Variante mit $n - i - 1$

```
prozedur bubbleSort(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            wenn a[j]  
                > a[j + 1] dann  
  
                temp = a[j]  
                a[j] = a[j + 1]  
                a[j + 1] = temp  
            ende wenn  
        ende für  
    ende für  
endprozedur
```

این نسخه کلاسیک BubbleSort است: در هر پاس، همسایه‌ها ($a[j]$ و $a[j+1]$) مقایسه می‌شوند و در صورت لزوم جابه‌جا (swap) می‌شوند. در پایان هر پاس، بزرگ‌ترین مقدار به انتهای آرایه می‌رود.

1.2 Variante mit zwei Schleifen 0 bis n-2

```

prozedur bubbleSortEinfach(
    a: Array von Integer)

    n = a.length

    für i = 0
        bis n - 2

        für j = 0
            bis n - i - 2

                wenn a[j]
                    > a[j + 1] dann

                        temp = a[j]
                        a[j] = a[j + 1]
                        a[j + 1] = temp
                    ende wenn
        ende für j
    ende für i
endprozedur

```

در بعضی امتحان‌ها به جای $n - i - 2$ از 0 تا $n - 2$ استفاده می‌شود (کمتر بهینه است اما ساده‌تر است).

2. BubbleSort für Objekte – مرتب‌سازی آبجکت‌ها

2.1 Sortieren nach getBelegung()

```
prozedur bubbleSortRaeume(  
    raeume: Array von Raum)  
  
    n = raeume.length  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            wenn raeume[j]  
                .getBelegung()  
                > raeume[j + 1]  
                    .getBelegung() dann  
  
                temp = raeume[j]  
                raeume[j] = raeume[j + 1]  
                raeume[j + 1] = temp  
            ende wenn  
        ende für i  
    ende für j  
endprozedur
```

اینجا به جای مقایسهٔ مستقیم اعداد، از متد `getBelegung()` استفاده می‌کنیم. الگو همان BubbleSort است، فقط روی ویژگی آبجکت کار می‌کند.

2.2 Sortieren nach getPreis()

```

prozedur bubbleSortArtikel(
    artikel: Array von Artikel)

n = artikel.length

für i = 0
    bis n - 2

        für j = 0
            bis n - i - 2

                wenn artikel[j]
                    .getPreis()
                    > artikel[j + 1]
                        .getPreis() dann

                    temp = artikel[j]
                    artikel[j] = artikel[j + 1]
                    artikel[j + 1] = temp
                ende wenn
            ende für j
        ende für i
    ende prozedur

```

3. Generischer BubbleSort – List<T> + Function<T>

3.1 Vergleich über Function<T>

```
void bubbleSort(  
    liste: List<T>,  
    f: Function<T>)  
  
    n = liste.size()  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            // f(a, b) > 0 ⇒ a "größer" als b  
            wenn f(  
                liste.get(i),  
                liste.get(j + 1))  
                > 0 dann  
  
                T r = liste.get(j)  
  
                liste.set(  
                    i,  
                    liste.get(j + 1))  
  
                liste.set(  
                    i + 1,  
                    r)  
            ende wenn  
        ende برای  
    ende برای  
end void
```

در این نسخه f مشخص می‌کند کدام عنصر بزرگ‌تر است. اگر $f(a, b) > 0$ باشد یعنی a باید بعد از b قرار بگیرد و جابه‌جایی انجام می‌شود. این الگو در امتحان‌های جدید بسیار رایج است.

4. BubbleSort absteigend – مرتب‌سازی نزولی

```
prozedur bubbleSortAbsteigend(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 2  
  
            für j = 0  
                bis n - i - 2  
  
                    wenn a[i]  
                        < a[j + 1] dann  
  
                        temp = a[i]  
                        a[i] = a[j + 1]  
                        a[j + 1] = temp  
                    ende wenn  
                ende برای  
            ende برای  
        endprozedur
```

فقط جهت مقایسه برعکس شده است (علامت > به جای <)، یعنی آرایه به صورت نزولی مرتب می‌شود.

5. BubbleSort für Strings – رشته‌ها

5.1 Alphabetisch (A–Z)

```
prozedur bubbleSortNamen(  
    namen: Array von String)  
  
    n = namen.length  
  
    für i = 0  
        bis n - 2  
  
            für j = 0  
                bis n - i - 2  
  
                    wenn namen[j]  
                        > namen[j + 1] dann  
                        // lexikografischer Vergleich  
  
                        temp = namen[j]  
                        namen[j] = namen[j + 1]  
                        namen[j + 1] = temp  
                    ende wenn  
            ende für j  
        ende für i  
    endprozedur
```

5.2 Nach Länge (kürzere zuerst)


```

prozedur bubbleSortNamenNachLaenge(
    namen: Array von String)

    n = namen.length

    für i = 0
        bis n - 2

            für j = 0
                bis n - i - 2

                    wenn namen[j].length()
                        > namen[j + 1]
                            .length() dann

                                temp = namen[j]
                                namen[j] = namen[j + 1]
                                namen[j + 1] = temp
                    ende wenn
            ende für j
        ende für i
    endprozedur

```

6. Optimierter BubbleSort – با Flag (Early-Stop)

```
prozedur bubbleSortOptimiert(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 2  
  
        getauscht = false  
  
        für j = 0  
            bis n - i - 2  
  
                wenn a[i]  
                    > a[j + 1] dann  
  
                        temp = a[i]  
                        a[i] = a[j + 1]  
                        a[j + 1] = temp  
  
                        getauscht = true  
                ende wenn  
        ende برای  
  
        wenn getauscht = false dann  
            // keine Tausche mehr ⇒ fertig  
            break  
        ende wenn  
    ende ،  
endprozedur
```

اگر در یک پاس هیچ جابه‌جایی (swap) انجام نشود، آرایه از قبل مرتب است و حلقه می‌تواند زودتر متوقف شود. این بهینه‌سازی فقط در BubbleSort معنی دارد.

7. CocktailSort – BubbleSort دوطرفه

```

prozedur cocktailSort(
    a: Array von Integer)

links = 0
rechts = a.length - 1

wiederhole

    getauscht = false

    // links → rechts
    für i = links
        bis rechts - 1

            wenn a[i]
                > a[i + 1] dann

                    temp = a[i]
                    a[i] = a[i + 1]
                    a[i + 1] = temp

                    getauscht = true
            ende wenn
        ende برای

rechts = rechts - 1

    wenn getauscht = false dann
        break
    ende wenn

getauscht = false

// rechts → links
für i = rechts
    bis links + 1
        rückwärts

            wenn a[i - 1]
                > a[i] dann

                    temp = a[i - 1]
                    a[i - 1] = a[i]
                    a[i] = temp

```

```
        getauscht = true
    ende wenn
    ende برای

    links = links + 1

bis getauscht = false
endprozedur
```

CocktailSort نسخهٔ دوطرفهٔ BubbleSort است: یک بار از چپ به راست و یک بار از راست به چپ می‌رود.

8. Fehler im BubbleSort – خطایابی در BubbleSort

8.1 Fehlerhafte Version (Beispiel)

```
prozedur sort(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 1  
  
        für j = 0  
            bis n - 1  
  
                wenn a[j + 1]  
                    > a[j] dann    // Vergleich verkehrt  
  
                    temp = a[j]  
                    a[j + 1] = temp    // Tausch unvollständig  
                ende wenn  
            ende برای  
        ende برای  
    endprozedur
```

8.2 Korrigierte Standard-Version

```

prozedur sort(
    a: Array von Integer)

    n = a.length

    für i = 0
        bis n - 2

        für j = 0
            bis n - i - 2

                wenn a[i]
                    > a[j + 1] dann

                        temp = a[i]
                        a[i] = a[j + 1]
                        a[j + 1] = temp
                ende wenn
        ende für j
    ende für i
endprozedur

```

اشتباهات کلاسیک در BubbleSort: مرزهای اشتباه حلقه‌ی j ، شرط مقایسه برعکس ($>$ به جای $<$)، و جابه‌جایی ناقص (فقط یک طرف).

9. Varianten mit Function<T> – مقایسه جنریک

9.1 Aufsteigend

```
void bubbleSort(  
    liste: List<T>,  
    f: Function<T>)  
  
    n = liste.size()  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            wenn f(  
                liste.get(i),  
                liste.get(j + 1))  
                > 0 dann  
  
                T r = liste.get(j)  
  
                liste.set(  
                    i,  
                    liste.get(j + 1))  
  
                liste.set(  
                    j + 1,  
                    r)  
  
            ende wenn  
        ende برای  
    ende برای  
end void
```

9.2 Absteigend


```

void bubbleSortAbsteigend(
    liste: List<T>,
    f: Function<T>)

n = liste.size()

für i = 0
    bis n - 2

    für j = 0
        bis n - i - 2

        // für absteigend:
        // wenn linkes Element "kleiner" ist ⇒ tauschen
        wenn f(
            liste.get(j),
            liste.get(j + 1))
            < 0 dann

            T r = liste.get(j)

            liste.set(
                j,
                liste.get(j + 1))

            liste.set(
                j + 1,
                r)
        ende wenn
    ende für j
ende für i
end void

```

10. Wichtige Theoriepunkte zu BubbleSort

- Komplexität: $O(n^2)$ im Durchschnitt und im Worst Case.
- Stabil: relative Reihenfolge gleicher Elemente bleibt erhalten.
- Einfach zu implementieren, aber ineffizient für große Datenmengen.
- Optimierbar mit Flag (Abbruch, wenn kein Tausch).
- In Prüfungen oft mit Objekten und `Function<T>` kombiniert.

نکات تئوری مهم:

- پیچیدگی زمانی: $O(n^2)$ در حالت متوسط و بدترین حالت.
 - الگوریتم پایدار (Stable) است: ترتیب عناصر مساوی حفظ می‌شود.
 - برای آموزش و امتحان ساده است، ولی برای داده‌های بزرگ کارایی خوبی ندارد.
 - با فلگ (getauscht) می‌توان آن را بهینه کرد.
 - در امتحان‌ها معمولاً با آبجکت‌ها و `Function<T>` می‌آید.
-

11. Prüfungsähnliche Übungen – تمرین‌های شبیه GA2

Übung 1 – Standard-BubbleSort (Integer)

Implementieren Sie eine Prozedur, die ein Array von ganzen Zahlen aufsteigend mit BubbleSort sortiert.

```
prozedur sortiere(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            wenn a[i]  
                > a[j + 1] dann  
  
                temp = a[i]  
                a[i] = a[j + 1]  
                a[j + 1] = temp  
            ende wenn  
        ende برای  
    ende ب اء  
endprozedur
```

Übung 2 – BubbleSort für Objekte (Raum)

Sortieren Sie ein Array von `Raum` -Objekten aufsteigend nach Belegung.

```
prozedur sortiereRaeume(  
    raeume: Array von Raum)  
  
    n = raeume.length  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            wenn raeume[i]  
                .getBelegung()  
                > raeume[j + 1]  
                    .getBelegung() dann  
  
                temp = raeume[i]  
                raeume[i] = raeume[j + 1]  
                raeume[j + 1] = temp  
            ende wenn  
        ende برای  
    ende برای  
endprozedur
```

Übung 3 – BubbleSort mit Function<T>

Implementieren Sie `void sort(List<T> liste, Function<T> f)`, die die Liste mit BubbleSort sortiert.

```
void sort(  
    liste: List<T>,  
    f: Function<T>)  
  
    n = liste.size()  
  
    für i = 0  
        bis n - 2  
  
        für j = 0  
            bis n - i - 2  
  
            wenn f(  
                liste.get(i) .  
                liste.get(j + 1))  
                > 0 dann  
  
                T r = liste.get(j)  
  
                liste.set(  
                    i,  
                    liste.get(j + 1))  
  
                liste.set(  
                    i + 1,  
                    r)  
            ende wenn  
        ende برای  
    ende برای  
end void
```

Übung 4 – Optimierter BubbleSort mit Flag

Erweitern Sie BubbleSort so, dass bei einem bereits sortierten Array früher abgebrochen wird.

```
prozedur sortOptimiert(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 2  
  
        getauscht = false  
  
        für j = 0  
            bis n - i - 2  
  
                wenn a[j]  
                    > a[j + 1] dann  
  
                        temp = a[j]  
                        a[j] = a[j + 1]  
                        a[j + 1] = temp  
  
                        getauscht = true  
                ende wenn  
        ende für  
  
        wenn getauscht = false dann  
            break  
        ende wenn  
    ende für  
endprozedur
```

Übung 5 – Fehler im BubbleSort korrigieren

Im folgenden Algorithmus sind mehrere Fehler. Korrigieren Sie ihn.

```
prozedur sort(  
    a: Array von Integer)  
  
    n = a.length  
  
    für i = 0  
        bis n - 1  
  
        für j = 0  
            bis n - 1  
  
                wenn a[i + 1]  
                    > a[j] dann  
  
                    a[i+1] = a[i + 1]  
                    // temp fehlt, Vergleich verkehrt  
                ende wenn  
            ende برای  
        ende برای  
    endprozedur
```

Korrekte Lösung: Standard-BubbleSort wie in Kapitel 1.

Ende der Masterdatei – BubbleSort.

پایان فایل مستر BubbleSort.